HDOC Day-40 Activity

Menu-Driven C Program - String Operations Using User-Defined Functions and Pointers

Name	Gurveer Singh Cheema
SAF ID	590032925
Batch	15

Aim

To write a menu-driven C program that reads a student's first, middle, and last name and performs different string operations using separate user-defined functions. Pointers are used for call by reference wherever a function modifies a string or returns multiple results.

Header Files Used

stdio.h - input/output functions; **string.h** - strcpy(), strcat(), strlen(), strcmp(), strstr(); **ctype.h** - isalpha(), isdigit(), isspace(), tolower().

Operations

1. Combine first, middle and last names.
2. Find the length of the full name.
3. Copy the full name into another string.
4. Compare first and last names.
5. Display the name in reverse order.
6. Count vowels, consonants, digits and spaces.
7. Search for a word in the full name.
8. Exit.

Algorithm

1. Read first name, middle name and last name.
2. Display a menu inside a do-while loop.
3. Read the user's choice.
4. Use switch-case to call the required user-defined function.
5. Use pointers for length, comparison result, character counts and search result.
6. Repeat the menu until the user selects Exit.

C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void combineNames(const char firstName[], const char middleName[],
                  const char lastName[], char fullName[])
{
    strcpy(fullName, firstName);
    strcat(fullName, " ");
    strcat(fullName, middleName);
    strcat(fullName, " ");
    strcat(fullName, lastName);
}

void findLength(const char fullName[], size_t *length)
```

```
{
    *length = strlen(fullName);
}

void copyName(const char source[], char destination[])
{
    strcpy(destination, source);
}

void compareNames(const char firstName[], const char lastName[],
                  int *comparisonResult)
{
    *comparisonResult = strcmp(firstName, lastName);
}

void reverseNameOrder(const char firstName[], const char middleName[],
                      const char lastName[], char reverseName[])
{
    strcpy(reverseName, lastName);
    strcat(reverseName, " ");
    strcat(reverseName, middleName);
    strcat(reverseName, " ");
    strcat(reverseName, firstName);
}

void countCharacters(const char fullName[], int *vowels,
                     int *consonants, int *digits, int *spaces)
{
    int i;
    char ch;

    *vowels = *consonants = *digits = *spaces = 0;

    for (i = 0; fullName[i] != '\0'; i++)
    {
        ch = tolower((unsigned char)fullName[i]);

        if (isalpha((unsigned char)ch))
        {
            if (ch == 'a' || ch == 'e' || ch == 'i' ||
                ch == 'o' || ch == 'u')
                (*vowels)++;
            else
                (*consonants)++;
        }
        else if (isdigit((unsigned char)ch))
            (*digits)++;
        else if (isspace((unsigned char)ch))
            (*spaces)++;
    }
}

void searchWord(const char fullName[], const char word[],
                int *searchResult)
{
    if (strstr(fullName, word) != NULL)
        *searchResult = 1;
    else
        *searchResult = 0;
}

int main()
{
    char firstName[50], middleName[50], lastName[50];
    char fullName[160], copiedName[160], reverseName[160], word[50];
    int choice, comparisonResult, vowels, consonants;
    int digits, spaces, searchResult;
    size_t length;

    printf("Enter first name: ");
    scanf("%49s", firstName);
    printf("Enter middle name: ");
    scanf("%49s", middleName);
    printf("Enter last name: ");
    scanf("%49s", lastName);

    combineNames(firstName, middleName, lastName, fullName);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Display full name\n");
        printf("2. Find length of full name\n");
        printf("3. Copy full name\n");
        printf("4. Compare first and last name\n");
```

```
        printf("5. Display name in reverse order\n");
        printf("6. Count vowels, consonants, digits and spaces\n");
        printf("7. Search a word in full name\n");
        printf("8. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                combineNames(firstName, middleName, lastName, fullName);
                printf("Full Name: %s\n", fullName);
                break;

            case 2:
                findLength(fullName, &length);
                printf("Length of full name: %zu\n", length);
                break;

            case 3:
                copyName(fullName, copiedName);
                printf("Copied Name: %s\n", copiedName);
                break;

            case 4:
                compareNames(firstName, lastName, &comparisonResult);
                if (comparisonResult == 0)
                    printf("First and last names are equal.\n");
                else
                    printf("First and last names are different.\n");
                break;

            case 5:
                reverseNameOrder(firstName, middleName,
                                 lastName, reverseName);
                printf("Reverse Order: %s\n", reverseName);
                break;

            case 6:
                countCharacters(fullName, &vowels, &consonants,
                                &digits, &spaces);
                printf("Vowels: %d\n", vowels);
                printf("Consonants: %d\n", consonants);
                printf("Digits: %d\n", digits);
                printf("Spaces: %d\n", spaces);
                break;

            case 7:
                printf("Enter word to search: ");
                scanf("%49s", word);
                searchWord(fullName, word, &searchResult);

                if (searchResult)
                    printf("Word found in full name.\n");
                else
                    printf("Word not found in full name.\n");
                break;

            case 8:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice! Try again.\n");
        }

    } while (choice != 8);

    return 0;
}
```

Sample Output

```
Enter first name: Gurveer
Enter middle name: Singh
Enter last name: Cheema

----- MENU -----
1. Display full name
2. Find length of full name
3. Copy full name
4. Compare first and last name
5. Display name in reverse order
6. Count vowels, consonants, digits and spaces
7. Search a word in full name
8. Exit

Enter choice: 1
Full Name: Gurveer Singh Cheema

Enter choice: 2
Length of full name: 19

Enter choice: 3
Copied Name: Gurveer Singh Cheema

Enter choice: 4
First and last names are different.

Enter choice: 5
Reverse Order: Cheema Singh Gurveer

Enter choice: 6
Vowels: 7
Consonants: 10
Digits: 0
Spaces: 2

Enter choice: 7
Enter word to search: Singh
Word found in full name.

Enter choice: 8
Exiting program...
```

Function Explanation

combineNames() uses strcpy() and strcat() to build the complete name. **findLength()** uses strlen() and returns the result through a pointer. **copyName()** copies the full name with strcpy(). **compareNames()** uses strcmp() and returns the comparison result through a pointer.

reverseNameOrder() forms the order last-middle-first. **countCharacters()** scans the full name and returns four counts through pointers. **searchWord()** uses strstr() and returns 1 when the word is found and 0 otherwise.

Call by Reference Used

The addresses of variables are passed to functions using the **&** operator, for example **&length;**, **&vowels;**, and **&searchResult;**. Inside the function, the pointer is dereferenced using ***** so that the original variable can be modified.

Result

Thus, the menu-driven C program was implemented successfully using string library functions, separate user-defined functions, and pointers for call by reference.

Quick Viva Questions

1. What does strcpy() do? It copies one string into another.
2. What does strcat() do? It joins one string to the end of another.
3. What does strlen() return? The number of characters in a string excluding the null character.
4. What does strcmp() return? Zero when two strings are equal; a non-zero value when they differ.

5. What does strstr() do? It searches for one string inside another string.
6. Why are pointers used here? They allow functions to modify original variables and return multiple results.
7. What is the purpose of ctype.h? It provides character-testing and conversion functions such as isalpha(), isdigit(), isspace(), and tolower().